Bonus Case

ANSWER KEY

```
>>> print(0.1 + 0.2)
0.30000000000000004
```

Finished all four cases? These go a step further. Use what you learned about bits, rounding and colors.

1. Python prints `0.30000000000000004` for `0.1 + 0.2`. Explain why, using what you learned in Case 1.

Round-off error. 0.1 and 0.2 can't be built exactly from binary place values (½, ¼, ⅛, ...); they repeat forever, so the computer stores the closest values it can. The tiny errors add up, so the sum is slightly off from 0.3.

---

2. A 6-bit system has the place values below. What is the closest value to **12.6** it can store? Fill in the bits.

Place value	8	4	2	1	½	¼	Value
Store 12.6	1	1	0	0	1	0	12.5

How far off is it? **0.1** *Check both neighbors: 12.5 is 0.1 away, 12.75 is 0.15 away.*

3. A 10 × 10 image can use 64 different colors. What is the smallest possible size in bits, ignoring the header? Show your work.

Step 1: 64 colors → 6 bits per pixel ($2^6 = 64$)
Step 2: 10 × 10 = 100 pixels × 6 bits = 600 bits (75 bytes)

4. Design challenge: invent a 2-bit-per-pixel grayscale code (4 shades). Write your key, then encode a 3 × 3 image of your choice.

Answers vary. One example:

- **00 = black**
- **01 = dark gray**
- **10 = light gray**
- **11 = white**

11	00	11
00	01	00
11	00	11

```
11 00 11
00 01 00
11 00 11
```

= 9 pixels × 2 bits = 18 bits

Check: every pixel uses exactly 2 bits, and every code in the image matches the key.

KEY IDEA A fixed number of bits always means a **limit**: on how big, how precise, or how colorful the data can be.